

Instructions for MA4

This document contains the material for the fourth module (M4) in the course Computer Programming II. Concepts covered are parsing with “recursive descent”, exceptions, and function objects.

In this module, you will write a calculator which reads and calculates arithmetic expressions. You will get a very simple version to start with. You will then add several features to this version. The description of this mandatory assignment can be found in the yellow boxes below.

The following files are associated with the module:

1. [MA4.py](#) A skeleton of the program that you are going to write and upload in the end.
2. [MA4tokenizer.py](#) A helper class that you have to use.
3. [MA4init.txt](#) A file with test data that is read automatically when the program starts. You can extend it with more test cases for the features that you implement.
4. [MA4_test.py](#): Unit-tests. Run this program when you think that you have completed all tasks!
5. [MA4micro.py](#) A small calculator that executes simple operations.

Follow the instruction below when preparing your solutions:

1. When you're done:
 - Check that the complete unit test and test data file works!
 - Make sure that you understand how exception handling works! Stepping in a debugger might help!
 - Demonstrate your solution orally to a TA, answer any questions.
 - Upload the file [MA4.py](#) within a zip archive in Studium under MA4.
 - **Write a comment in Studium to identify which of the tasks in the final yellow box that you choose to complete.**
2. You may *not* use packages other than those already imported.
3. Try to implement and test one feature at a time! If you want to run the unit test before every function is in place, comment out tests for what you haven't implemented and enable them gradually.
4. Handle the functions that are provided in the calculator syntax by using a dictionary mapping calculator syntax function names to Python *function objects*. Adding a new function should only require updating this dictionary (and implementing the corresponding function object).

Note that your solution should just consist of **one** Python file. Zip it for a simpler submission process. We do not accept other formats (Word files, PDF files, Jupyter notebooks, ...). No submissions should be done over e-mail.

Important note:

You may collaborate with other students, search the Internet, and get a general overview using AI tools, but you must write and be able to explain the code you actually present. You may not copy code, either from other students, automated tools, or from the Internet, except from those places explicitly pointed out in this module.

Changing variable names and similar superficial modifications does not count as writing your own code. Also, aiding somebody to cheat (for example, if somebody hands in your code as their own) is not permitted and will be reported.

Since these assignments are included as part of the course examination, we are obliged to report blatant violations of these rules to the disciplinary board.

All course material that we provide is copyrighted and may not be spread.

Please report any errors or unclear passages in this document to the course teacher using the feedback form.

Syntax analysis with recursive descent

Recursive methods are well-suited to processing recursively structured data. An example of this is ordinary arithmetic expressions. Consider e.g. the following expression

$$1 + (2.3 + 4) \cdot 3.42 + 0.34 \cdot (1.2 + 13 \cdot 0.7)$$

There are a number of rules for how this should be interpreted, for example “multiplication before addition”, “parentheses first” and “from left to right when precedence is equal”. It is not easy (but doable) to implement these rules in a program which reads and interprets an expression.

However, we can define expressions in a more structured way. This structure will implicitly also state the precedence rules:

An *expression*

is a sequence of one or more *terms*, each separated by a plus or minus character.

A *term*

is a sequence of one or more *factors*, each separated by a multiplication or division character.

A *factor*

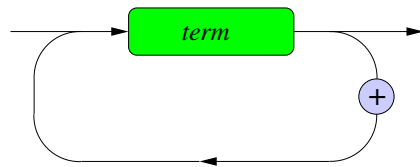
is either a *number*, or an *expression* surrounded by parentheses.

Note that this definition is indirectly recursive.

Initially we only consider syntactically correct expressions. For the sake of simplicity in the demo code, we will limit ourselves for the time being to addition, multiplication and parentheses operations. This does not change the structure — only the details of the code.

The definition can be illustrated graphically with the following *syntax diagrams*. There is *pseudocode* to the right of the diagrams describing how the diagram can be translated into code. Here, we combine **if** and **while** statements with descriptions as comments for parts to be specified later.

expression

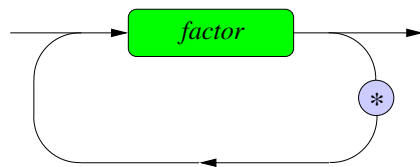


```

1 def expression():
2     sum = term()
3     while # next is '+':
4         # Get passed '+'
5         sum += term()
6     return sum

```

term

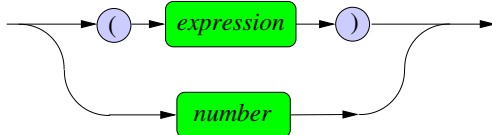


```

1 def term():
2     product = factor()
3     while # next is '*':
4         # Get passed '*'
5         product *= factor()
6     return product

```

factor



```

1 def factor():
2     if # next is '(':
3         # Get passed '('
4         result = expression()
5         # Get passed ')'
6     else:
7         result = # read number
8     return result

```

We have not drawn a diagram for *numbers* but assume that there is a piece of code that can extract a number using digits, decimal point etc.

Note the recursion again: an *expression* is defined by *terms*, which are defined by *factors* which, maybe, are defined by an *expression*.

All diagrams contain “junctions”. It is the next character (plus, times or parentheses) which decides which path we should choose. If, for example, in the *expression* there is a plus character after the first term, we loop back and consume another term. If there is no plus character, the expression is complete **no matter what follows** — that’s not our problem at this level of the syntax!

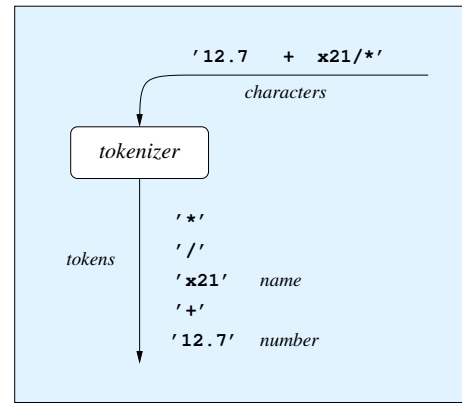
The input to the program is basically a sequence of characters (digits 0-9 as well as ‘.’, ‘+’, ‘*’, ‘(’ and ‘)’). We want to handle *numbers*, not individual digits, decimal point etc. For this we will use a so-called *tokenizer*.

The tokenizer

A *tokenizer* is used to group individual characters into larger units such as words and numbers, so-called *tokens*. Different applications have different rules for how the tokens are formed. Of course, a tokenizer for programming code has different rules from a tokenizer for natural language processing.

The Python module `tokenize` is useful for our purposes. To simplify its use, we provide a wrapper class `TokenizeWrapper`. An object from the wrapper class is tied to a character string (such as an input line) and produces a sequence of tokens. There is a *current token*, methods to get information about this token, and a method for advancing to the next token.

This figure exemplifies how a tokenizer receives the string '12.7 + x21/*' with 13 characters and produces a sequence of 5 different string tokens.



The most important methods in the `TokenizerWrapper` are:

<code>TokenizerWrapper(line)</code>	Creates a tokenizer object, ties it to <code>line</code> , sets the first current token
<code>next()</code>	Advances to the next token (number, character, word, ...)
<code>has_next()</code>	Returns <code>True</code> if there are more tokens on the line, otherwise <code>False</code>
<code>get_current()</code>	Returns the current token as a string
<code>get_previous()</code>	Returns the previous token as a string
<code>is_number()</code>	Returns <code>True</code> if the current token is a number, otherwise <code>False</code> .
<code>is_name()</code>	Returns <code>True</code> if the current token is a name, otherwise <code>False</code> .
<code>is_at_end()</code>	Returns <code>True</code> if end of line otherwise <code>False</code> .
<code>is_newline()</code>	Returns <code>True</code> if current token is a newline character otherwise <code>False</code> .

Note that `next()` is the *only* method that advances the tokenizer. The other methods just provide information about the current or previous token.

Here is a small demo code. The output is shown in the adjacent pane.

hej	NAME
hopp	NAME
+	OPERATOR
234	NUMBER
*	OPERATOR
26.4	NUMBER
=	OPERATOR
FINAL NEWLINE	

```

1 def main():
2     line = 'hej hopp + 234 * 26.4 =\n'
3     wtok = TokenizeWrapper(line)
4     while wtok.has_next():
5         print(wtok.get_current(), end='\t')
6         if wtok.is_name():
7             print('NAME')
8         elif wtok.is_number():
9             print('NUMBER')
10        elif wtok.is_newline():
11            print('NEWLINE')
12        else:
13            print('OPERATOR')
14        wtok.next()
15    print(wtok.get_current())
16    if wtok.is_newline():
17        print('FINAL NEWLINE')

```

Using these methods, we can express the description in comments in our previous example as actual Python code. We will then get the following simple calculator:

```

1 def expression(wtok):
2     result = term(wtok)
3     while wtok.get_current() == '+':
4         wtok.next()          # bypass +
5         result = result + term(wtok)
6     return result
7
8 def term(wtok):
9     result = factor(wtok)
10    while wtok.get_current() == '*':
11        wtok.next()          # bypass *
12        result = result * factor(wtok)
13    return result
14
15 def factor(wtok):
16     if wtok.get_current() == '(':
17         wtok.next()          # bypass (
18         result = expression(wtok)
19         wtok.next()          # bypass )
20     else:                     # should be a number
21         result = float(wtok.get_current())
22         wtok.next()          # bypass the number
23     return result
24
25 def main():
26     print("Very simple calculator")
27     while True:

```

```

28     line = input("Input : ")
29     wtok = TokenizeWrapper(line)
30     if wtok.get_current() == 'quit':
31         break
32     else:
33         res = expression(wtok)
34         print('Result: ', res)
35     print("Bye!")
36
37 if __name__ == "__main__":
38     main()
39

```

What can go wrong?

```

3*2 ++ 1
2-3
-1
1 2 3
1**2
2*(1+3-+4

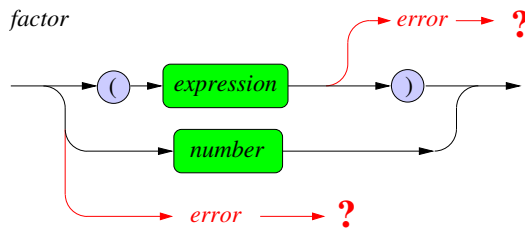
```

The program above is included in a file named `MA4micro.py`. Now, you can also try these deliberately incorrect lines, relative to what the current calculator supports:

Try to understand the output!

There are two places in the code where things can go wrong, and both are when dealing with factors:

- 1) The factor begins neither with a left parenthesis nor with a number.
- 2) There is no matching closing parenthesis after *expression*.



The errors can easily be detected by the code:

```

1 def factor(wtok):
2     if wtok.get_current() == '(':
3         wtok.next()          # bypass (
4         result = expression(wtok)
5         if wtok.get_current() == ')':
6             wtok.next()      # bypass )
7         else:
8             pass # What shall we do here
9     elif wtok.is_number():
10        result = float(wtok.get_current())
11        wtok.next()          # bypass the number
12    else:
13        pass # What shall we do here?
14    return result
15

```

It is possible to produce an error message here but how should the program continue? The function is expected to return a number that is to be used higher up in the code. Once an error has been detected, it is probably pointless to continue calculating the expression. However, we do not want the program to exit but to print the error message, ignore the rest of the expression and continue with a new expression on a new line.

To handle such situations, *exceptions* are well-suited.

Error handling in the calculator

To handle syntax errors in the calculator, we will make our *own* exception class. It must be written as a so-called subclass of the built-in class `Exception`. Exception classes are easy to write. In this case we only need:

```

1 class CalculatorSyntaxError(Exception):
2     def __init__(self, arg):
3         self.arg = arg
4         super().__init__(self.arg)

```

Thus, the class has only an `__init__` constructor, which receives and saves a message. The word `Exception` in parentheses in line 1 indicates that this is a subclass of the `Exception` class.

To trigger an exception, we use the keyword `raise`. Let's use `raise` in the `factor` function and handle the exceptions in `main`:

```

1 def factor(wtok):
2     if wtok.get_current() == '(':
3         wtok.next()          # bypass (
4         result = expression(wtok)
5         if wtok.get_current() == ')':
6             wtok.next()      # bypass )
7     else:

```

```

8         raise CalculatorSyntaxError("Expected ')')")
9     elif wtok.is_number():
10         result = float(wtok.get_current())
11         wtok.next()          # bypass the number
12     else:
13         raise CalculatorSyntaxError('Expected number or (')
14     return result
15
16 def main():
17     print("Very simple calculator")
18     while True:
19         line = input("Input : ")
20         wtok = TokenizeWrapper(line)
21         try:
22             if wtok.get_current() == 'quit':
23                 break
24             else:
25                 result = expression(wtok)
26                 if wtok.is_at_end():
27                     print('Result: ', result)
28                 else:
29                     raise CalculatorSyntaxError('Unexpected token')
30         except CalculatorSyntaxError as se:
31             print("*** Syntax: ", se.arg)
32             print(f"The error occurred at token '{wtok.get_current()}'" +
33                   f" just after token '{wtok.get_previous()}'")
34         except TokenError:
35             print('*** Syntax: Unbalanced parentheses')
36     print('Bye!')

```

We have also added handling of a `TokenError` that can occur in case of unbalanced parentheses. In the test file, a `CalculatorSyntaxError` is expected when unbalanced parentheses occur. Feel free to use `TokenError` instead (and you may change the test file).

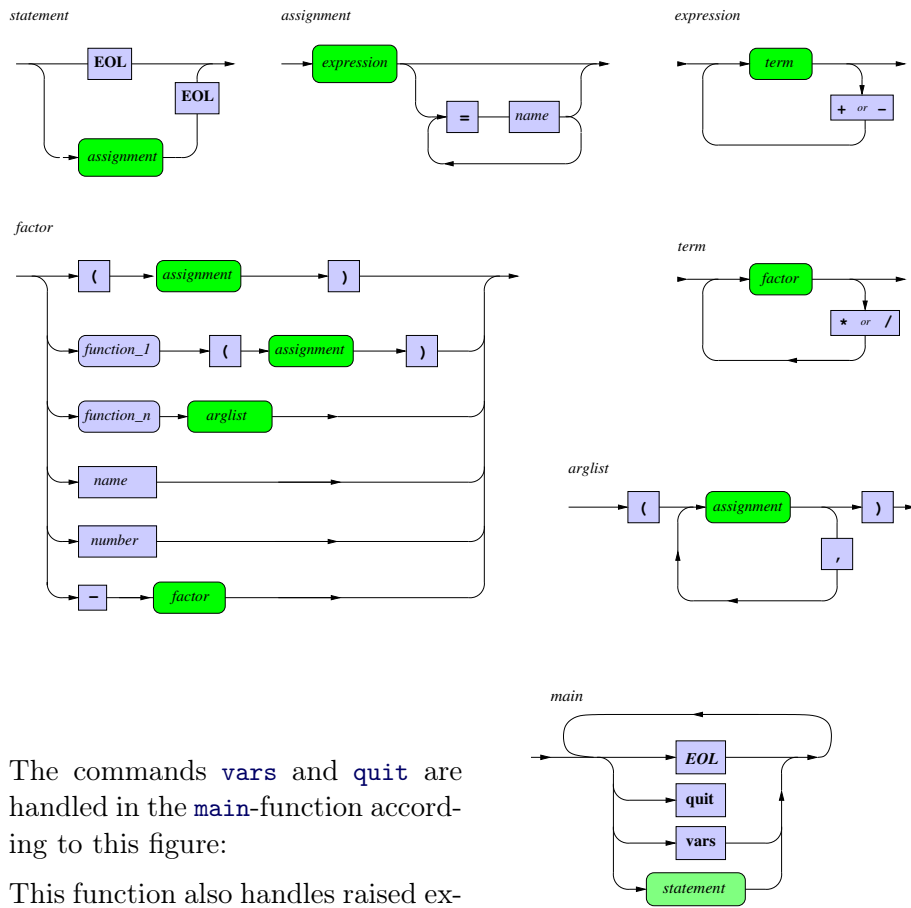
Mandatory assignment: Calculator

We start by exemplifying a run of the program and the complete syntax diagrams:

```
Input : 3-1+3-1/2          # Float type
Result: 4.5
Input : 1 - (5-2*2)/(1+1) - (-2 + 1)  # Float type
Result: 1.5
Input : sin(3.14159265)          # Standard functions
Result: 3.5897930298416118e-09
Input : cos(PI)                  # Predefined constant
Result: -1.0
Input : log(exp(4*0.5 - 1))      # Standard functions
Result: 1.0
Input : 1 + 2 + 3 = x           # Variable. Assignment applied RIGHTWARDS!
Result: 6.0
Input : x/2 + x
Result: 9.0
Input : (1=x) + sin(2=y)         # Assignments
Result: 1.9092974268256817
Input : vars                     # Prints variable values
  E      : 2.718281828459045
  PI     : 3.141592653589793
  ans    : 1.9092974268256817
  x      : 1.0
  y      : 2.0
Input : 1+2
Result: 3.0
Input : 2*ans + 5                # Result of last computation
Result: 11.0
Input : ans
Result: 11.0
Input : 1+z                      # Undefined variable
*** Evaluation error: Undefined variable: 'z'
Input : 1 + fib(5)
Result: 6
Input : fac(ans)                 # Factorial. Note integer
Result: 720
Input : fac(2.5)                 # Illegal argument
*** Evaluation error: Argument to fac is 2.5. Must be integer >= 0
Input : fac(40)                  # Larger integer
Result: 8159152832478977343456112695961158942720000000000
Input : 2 ++ 4*ans/0             # Syntax error before evaluation error
*** Syntax error: Expected number, word or '('
The error occurred at token '+' just after token '+'
Input : ans/0 + * x              # Evaluation error before syntax error
*** Evaluation error: Division by zero
```

Syntax diagrams

The syntax of the expressions is defined by the following diagrams:



The commands `vars` and `quit` are handled in the `main`-function according to this figure:

This function also handles raised exceptions.

EOL means End Of Line.

The task is to complement `MA4.py` to handle several operations and tasks as follows

1. The program should be able to handle expressions with constants, variables, the arithmetic operators `+`, `-`, `*`, `/`. The usual precedence rules apply and when used, parentheses should change the calculation order.
2. According to the diagram, unary minus should be implemented. It returns the negation of the subsequent factor. For example, $-(1+2) = -3$
3. The predefined variable `ans` should contain the value of the last calculated complete expression. This variable can then be used in the next expression. To keep track of the values of variables, a dictionary should be used. The dictionary `variables` is created in the `main` function and is then passed as an argument to all parser functions. Example:

```
Input : 1+1
Result: 2.0
Input : ans
Result: 2.0
Input : exp(2)
Result: 7.38905609893065
Input : ans
Result: 7.38905609893065
Input : ans + 3
Result: 10.38905609893065
Input : 3 + ans
Result: 13.38905609893065
```

4. Add the constants `PI` and `E` to the dictionary before expression processing starts. This makes the handling as uniform as possible. You can use the values from `math.e` and `math.pi`.
5. Variable assignment must be made *from left to right*, the opposite direction from Python syntax.

Example: The expression

```
1+2*3 = y
```

will give the value 7 to `y`.

6. Variable assignment should be allowed within subexpressions.

Example:

```
(2=x) + (3=y=z) = a
```

will give values to `x` (2), `y` (3), `z` (3) och `a` (5).

7. The command `vars` shows all stored variables with values and the command `quit` ends the run. Example:

```
Input : vars
ans    : 13.38905609893065
E      : 2.718281828459045
PI     : 3.141592653589793
x      : 1.0
y      : 2.0
Input : quit
Bye!
```

Functions

8. In addition to the functions `sin`, `cos`, `exp`, `log`, the following functions must be available:

Function	Meaning	Example
<code>fac(n)</code>	$n!$ Integer argument and result.	<code>fac(0) = 1, fac(1) = 1, fac(3) = 6</code>
<code>fib(n)</code>	n -th Fibonacci number	<code>fib(0) = 0, fib(1) = 1, fib(2) = 1, fib(3) = 2</code>

Functions should be implemented using a dictionary with *function names* as keys and *function objects* as values. It is a good idea to use different dictionaries for functions of one argument *function_1*, and many arguments, *function_n*, if you implement that feature (see the last exercise).

These dictionaries could be placed either locally in `factor` or on the top level as global constants.

To add a new function, there should be *no changes* needed to the code, beyond adding a new key-value pair to the dictionary. The function definition itself must of course be included.

As the syntax diagrams show, there are two different groups of functions:

1. Those with exactly one argument: `sin`, `cos`, `exp`, `log`, `fac` which are called *function_1* in the diagrams.
2. Those with several arguments: `max` and `sum` which are called *function_n* in the diagrams.

This example demonstrates how function objects can be stored and transferred.

Note that the example deals with *function objects*, not the return values from functions!

```

1 from math import sqrt, log
2
3 def demo(f, x):
4     return f(x)
5
6 print(demo(sqrt, 4))
7 print(demo(log, 1))
8
9 foo = sqrt
10 print(foo(9))           # Prints the square root of 9
11
12 d = {'f':sqrt, 'g':log}
13 print(d['f'](25))       # Prints the square root of 25

```

More about handling functions as objects can be found at Corey Schafer's [YouTube lesson](#)!

Error handling

9. The program must detect and diagnose errors. Example:

```
Input : 1++2
*** Syntax error: Expected number, word or '('
*** The error occurred at token '+' just after token '+'
Input : 1+-2
Result: -1.0
Input : 1--2
Result: 3.0
Input : 1**2
*** Syntax error: Unexpected token
*** The error occurred at token '**' just after token '1'
Input : 1/0
*** Evaluation error: Division by zero
Input : 2=y
Result: 2.0
Input : 1+2*y-4
Result: 1.0
Input : 1+2*k-4
*** Evaluation error: Undefined variable: k
Input : 1+2=3+4**x - 1/0
*** Syntax error: Expected variable after '='
*** The error occurred at token '3' just after token '='
Input : 1+2*(3-1 a
*** Syntax error: Unbalanced parentheses
*** The error occurred at token 'a' just after token '1'
Input : 1+2+3+
*** Syntax error: Expected number, word or '('
*** The error occurred at token '*EOL*' just after token '+'
Input : fac(2.5) # Illegal argument
*** Evaluation error: Argument to fac is 2.5. Must be integer >= 0
Input :
```

The user of the program can make two types of errors: *syntax errors* and *evaluation errors*. The expression `x+y` is an example of incorrect syntax while the expression `log(2*x-x-x)` is an example of an evaluation error (because the argument to `log` will be 0).

In the first expression the error can be detected *before* the summation is to be done. In the second case, the argument must be calculated before the error can be detected. Then, information about the current token is usually not relevant.

Errors are caught in the `main` function. When an error occurs, the rest of the line is ignored and a new expression is begun.

Coding hint:

Checking that the arguments are correct is probably best done in the individual functions (`log`, `fac`, ...) and *not* in the parser function `factor`.

Since all calculations are done with type `float`, it is the *value* and not the *type* that determines whether arguments to e.g. `fac` are correct or not. See the examples!

Include the error checks *from the beginning*! This greatly facilitates your own troubleshooting and testing of the program! It is good to implement this feature early on.

Syntax error

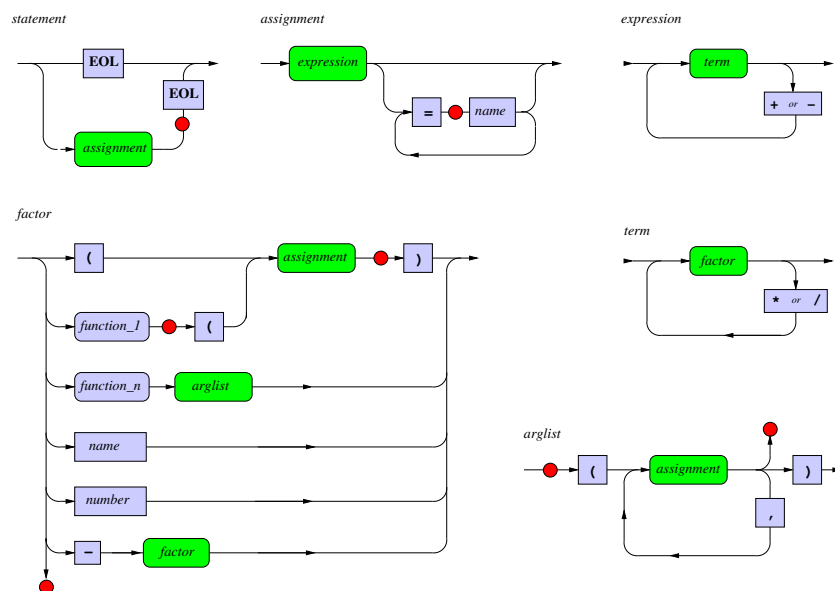
If the user writes an expression which does not match the syntax according to the syntax diagrams (e.g. `a ++ b` or `sin + 4` or `2 * 3) + 8`), it is a *syntax error*.

The program should detect such errors and deal with them with an exception of type `CalculatorSyntaxError`.

It can be tempting to look for errors everywhere but it only makes sense in a few places:

- In *assignment*: the equals sign is not followed by a variable name.
- In *factor*:
 1. The right parenthesis after *assignment* is missing.
 2. The left parenthesis after *function_1* is missing.
 3. None of the five allowed constructs is present, i.e. not a parenthesis, not a *function_1* or *function_n* name, not a number, not a minus sign and not a name.
- In *arglist*:
 1. The argument list does not start with a left parenthesis.
 2. An argument is not followed by a comma or a parenthesis.
- In *statement*: There are more items on the line.

The red circles indicate places to check the syntax.



In *expression* and *term*, nothing can go wrong at these levels. These methods should therefore not look for errors.

Evaluation errors

Even if an expression is syntactically correct, it may be impossible to evaluate. Examples of such errors:

- Division by 0
- Argument to `log` less than or equal to 0
- Incorrect type of argument (e.g. `fac(2.5)`)

These errors should be handled with the exception class `EvaluationError`. If it is an incorrect argument (i.e. all examples except the first) then the function name and the

argument value should be specified in the error message.

Program parts

Thus, the program will have the following components (with specified names):

- The class `TokenizerWrapper`. This class is given and you should not modify it in any way.
- A number of different functions which handle the parsing and calculations. There should be a function for each syntactic element (i.e. *assignment*, *expression*, *term*, *factor*, ...) These functions should exactly follow the syntax diagrams!
All must have exactly the two given parameters `wtok` (the tokenizer object) and `variables` (the variable dictionary).
All parser functions should return a number (`float` or `int`).
- Functions to evaluate: `fac`, `fib`, Of course, you can directly use functions built into Python (`cos`, `max`, ...)

Final exercise with multiple options

10. Finally, pick only one exercise to complete from the following list. **Write a comment in Studium to identify which of the tasks you have chosen to complete.**
 - Add a dictionary with functions accepting several arguments such as `max`, `min`, `sum`, `mean`. You may use the function `arglist` which returns a list of numbers.
 - Implement absolute value as an expression enclosed in vertical lines: $|-1| = 1$, $|3-5| = 2$.
 - Add `%` for the remainder from division (modulo). Same priority as `*` and `/`. Division by 0 must be handled for this operator as well.
 - Add `**` as a power operator. Undefined cases and cases resulting in complex numbers must be handled. Consider 0^{**-1} , $(-2)^{**0.5}$. Hint: the tokenizer represents `**` as a single token.